GeoPulse: Project Report

Real-Time Emergency Response & Resource Dispatch System

Student Name: Bhupendra Sirvi	**Degree:** B.Tech (Computer Science & Engg.)
Project Domain: Web & NoSQL Database	**GitHub:** github.com/Bhupendrasirvi/GeoPulse

1. Introduction

During natural disasters like flash floods, building collapses, or urban water-logging, the speed of response determines victim survival. Traditional municipal helplines often waste precious minutes because phone operators must manually write down addresses and search for available rescue squads.

GeoPulse is a fast, web-based emergency dispatch platform that directly connects citizens in danger with the nearest available rescue teams and shelters. By taking advantage of **MongoDB's built-in geospatial queries**, GeoPulse calculates true spherical distances in less than 20 milliseconds without needing complicated external software.

2. Project Objectives

- **Sub-20ms Location Search:** Find the nearest rescue team within 15 km in under 20 milliseconds using MongoDB's 2dsphere index.
- **Simple Citizen SOS Portal:** Allow citizens to transmit their exact GPS coordinates, emergency type, and number of people trapped with a single tap.
- **Autonomous 48-Hour Cleanup:** Use MongoDB's Time-To-Live (TTL) feature to auto-delete resolved alerts after 48 hours without needing manual cleanup scripts.
- **Smart Report Stacking:** Group duplicate calls from the same location into a single card with a counter (e.g. x3 reports) to prevent dispatcher screen clutter.
- **Local Backup Squad Creation:** Automatically commission a local neighborhood volunteer squad if no pre-registered team is within 15 km, ensuring zero victims are left unattended.
- **No Heavy Third-Party Queues:** Keep the entire architecture simple and lightweight using only Node.js, Express, and MongoDB without requiring Redis or Kafka.

3. Technologies Used

Component	Technology	Role in Project
Frontend UI	HTML5, Tailwind CSS, JS	Lightweight, dual-console interface (Citizen SOS + Dispatch Map)
Interactive Map	Leaflet.js + Esri Dark Canvas	High-contrast dark-mode map, live pins, tooltips, and dispatch vectors
Backend API	Node.js & Express.js	Handles REST endpoints and coordinates with MongoDB driver
Database	MongoDB (Local / Atlas)	Stores coordinates, runs $geoNear queries, and manages 48h TTL cleanup

4. How the System Works

A. Citizen SOS Workflow:

1. Citizen opens the portal and clicks *'Use My GPS'* or clicks on the map to drop their pin.
2. Citizen selects their emergency type (Medical, Flood, Fire, Supplies) and victim count.
3. Citizen clicks *'Transmit Emergency SOS Beacon'*. The alert is saved immediately into MongoDB.

B. Dispatcher Command Console Workflow:

1. Dispatcher sees a color-coded feed: Red for Action Needed, Blue for En Route, Green for Resolved.
2. Dispatcher clicks *'Match Unit'* (or presses 'M' key). The system queries MongoDB and draws a blue route vector to the nearest squad.
3. When the operation concludes, dispatcher clicks *'Resolve'* (or presses 'R' key) to trigger the 48h TTL auto-delete timer.

5. Database Design & Collections

MongoDB stores records as flexible BSON documents. GeoPulse uses 3 main collections:

Collection 1: 'incidents' (Sample Document)

```
{
  "id": "SOS-104",
  "caller": "Kavita Sharma",
  "phone": "+91 98141-89210",
  "category": "Medical / Trauma",
  "severity": "CRITICAL",
  "victims": 2,
  "note": "Diabetic patient having chest pain, road flooded.",
  "location": { "type": "Point", "coordinates": [75.7050, 31.2555] },
  "status": "OPEN",
  "assignedTo": null,
  "createdAt": "2026-10-15T14:30:00Z",
  "resolvedAt": null
}
```

Collection 2: 'responders' (Sample Document)

```
{
  "id": "R-PB1",
  "name": "LPU Campus Quick Response Squad",
  "type": "Rescue Team",
  "status": "Available",
  "contact": "+91 98761-11001",
  "location": { "type": "Point", "coordinates": [75.7045, 31.2560] }
}
```

6. Key Features & Code Implementation

1. Finding Nearest Rescue Unit ($geoNear Pipeline):

```
// Executed inside /api/match in server.js
const pipeline = [
  {
    $geoNear: {
      near: { type: "Point", coordinates: [lng, lat] },
      distanceField: "distanceMeters",
      spherical: true,
      maxDistance: 15000 // 15 km search radius
    }
  },
  { $limit: 1 }
];
const nearestTeam = await db.collection('responders').aggregate(pipeline).toArray();
```

2. Autonomous 48-Hour TTL Index Setup:

```
// Created once in server.js:
db.incidents.createIndex(
  { resolvedAt: 1 },
  { expireAfterSeconds: 172800 } // 172,800 seconds = 48 Hours
);
```

3. Keyboard Hotkeys for Fast Dispatch Operations:

- **[/]** : Jump to search box | **[J / K]** : Move up/down incident list
- **[M]** : Trigger nearest unit match | **[R]** : Mark current incident as resolved

7. REST API Endpoints

Method	Endpoint	Description
`GET`	`/api/incidents`	Fetch all active and resolved emergency incidents
`POST`	`/api/incidents`	Submit a new citizen SOS distress report into MongoDB
`GET`	`/api/responders`	List all emergency squads and their current locations
`POST`	`/api/match`	Run $geoNear to find and assign the closest team
`POST`	`/api/resolve`	Mark incident resolved and activate 48h TTL purge timer

8. How to Run the Project Locally

Step 1: Start MongoDB Service: net start MongoDB

Step 2: In geopulse-backend directory, run: node server.js

Step 3: Open geopulse-frontend/index.html in any web browser, or double-click start.bat.

9. Conclusion

GeoPulse successfully proves that an effective, real-time emergency dispatch system can be built without complicated, expensive infrastructure. By combining MongoDB's native geospatial capabilities with a responsive, easy-to-use map interface, the system connects victims to closest responders in sub-second time, helping save critical minutes during real-world crises.